Loan Origination System Analysis

Introduction

The loan origination process is a critical component of the financial services industry, involving multiple stages from lead generation to loan disbursement. However, financial institutions often face challenges such as high drop-off rates, lengthy processing times, and inefficient underwriting decisions. These inefficiencies can lead to lost revenue opportunities, increased operational costs, and a poor customer experience. This case study aims to address these issues by developing a data-driven approach to optimize the loan origination pipeline. By consolidating data from CRM systems, underwriting platforms, and credit bureaus, the goal is to enhance decision-making, improve approval rates, reduce processing times, and maximize conversion rates.

Objectives

- Improve conversion rates by identifying key bottlenecks in the loan application process and optimizing the customer journey.
- Enhance underwriting efficiency by leveraging data analytics to assess risk more accurately and streamline approval decisions.
- Reduce application decline rates by analyzing rejection reasons and refining lead targeting strategies.
- Optimize loan processing time by identifying inefficiencies in manual reviews and automating workflows where feasible.
- Increase customer retention and satisfaction by providing a seamless, faster loan approval process.
- Enable data-driven decision-making by integrating multiple data sources into a centralized model to improve operational insights.

Key Analysis Questions

1. What are the key conversion rates at each stage (lead, application, approval, disbursement), and where are the biggest drop-offs occurring?
2. Which factors contribute most to application declines, and how can decline rates be reduced through better risk assessment or alternative loan products?
3. How does loan processing time vary across different customer segments, and where can processing delays be minimized?

4. Which marketing and acquisition channels generate the most high-quality loan applicants, and how can marketing spend be optimized?
5. How do underwriting decisions correlate with repayment performance, and can predictive analytics improve risk evaluation?
6. What patterns emerge in approved vs. declined applications, and how can lenders enhance financial inclusion while maintaining portfolio quality?

Data Transformation Queries

```
--1. Merging into fact_loan_journey (Loan process tracking)

DROP TABLE IF EXISTS los_fact_loan_journey ;

CREATE TABLE IF NOT EXISTS los_fact_loan_journey AS

SELECT

    fl.lead_id, fl.customer_id,fl.lead_source,fl.lead_status,fl.created_date as lead_created_date,

    fa.application_id, fa.loan_id, fa.application_status_id, fa.loan_amount, fa.tenure,
fa.interest_rate AS

    applied_interest_rate,fa.applied_date as application_applied_date,fa.region_id,
fa.channel_id, fa.agent_id,

    fu.underwriting_id, fu.risk_score, fu.approval_status,

    fu.approved_amount,fu.requested_amount,fu.credit_score,fu.processing_time_days,
fu.decision_date,

    fd.disbursement_id, fd.disbursed_amount, fd.disbursement_date, fd.loan_start_date,
fd.loan_end_date, fd.repayment_term_months,

    fd.interest_rate AS final_interest_rate,

    fdec.decline_id, fdec.decline_reason_id, fdec.credit_score_at_decline,
fdec.income_at_decline,

    fdec.loan_amount_requested,fdec.decline_date


FROM fact_leads fl

JOIN fact_applications fa ON fl.customer_id = fa.customer_id

JOIN fact_underwriting fu ON fa.application_id = fu.application_id

JOIN fact_disbursements fd ON fa.application_id = fd.application_id
```

```
JOIN fact_declines fdec ON fa.application_id = fdec.application_id;


--2. Merging into dim_customer_details (Customer & Feedback details)
DROP TABLE IF EXISTS los_dim_customer_details ;
CREATE TABLE IF NOT EXISTS los_dim_customer_details AS
SELECT
    dc.customer_id, dc.first_name as cust_first_name, dc.last_name as cust_last_name, dc.dob as cust_DOB, dc.gender as cust_gender,
    dc.income as cust_income, dc.employment_status as cust_employment_status, dc.credit_score as cust_credit_score, dc.address as
    cust_address, dc.region_id,
    fc.feedback_id, fc.application_id, fc.agent_id, fc.feedback_score, fc.processing_time_satisfaction, fc.communication_satisfaction,
    fc.ease_of_application, fc.improvement_suggestions
FROM dim_customers dc
JOIN fact_customer_feedback fc ON dc.customer_id = fc.customer_id;


--3. Merging into dim_business_insights (Loan, Channel, Agent, Performance)
DROP TABLE IF EXISTS los_dim_business_insights ;
CREATE TABLE IF NOT EXISTS los_dim_business_insights AS
SELECT
    dl.loan_id, dl.loan_type, dl.interest_rate, dl.max_tenure, dl.min_amount, dl.max_amount,dl.eligibility_criteria,
    fa.applied_date,
    da.status_id, da.status_name, da.description AS status_description,
    dr.region_id, dr.country, dr.state, dr.city, dr.postal_code,
    dch.channel_id, dch.channel_name, dch.channel_type, dch.cost_per_acquisition,
    dd.reason_id, dd.reason_name, dd.risk_category, dd.mitigation_strategy,
    dag.agent_id, dag.first_name AS agent_first_name, dag.last_name AS agent_last_name, dag.assigned_channel_id, dag.experience_years,
```

```
    dag.performance_rating,
    fcp.performance_id, fcp.leads_generated, fcp.applications_submitted, fcp.conversion_rate, fcp.cost_per_lead, fcp.revenue_generated,
    fd.decline_date
FROM dim_loans dl
JOIN fact_applications fa ON dl.loan_id = fa.loan_id
JOIN dim_application_status da ON fa.application_status_id = da.status_id
JOIN dim_regions dr ON fa.region_id = dr.region_id
JOIN dim_channels dch ON fa.channel_id = dch.channel_id
JOIN dim_agents dag ON fa.agent_id = dag.agent_id
JOIN fact_channel_performance fcp ON dch.channel_id = fcp.channel_id
JOIN fact_declines fd ON fa.application_id = fd.application_id
JOIN dim_decline_reasons dd ON fd.decline_reason_id = dd.reason_id;
```

Key Performance Indicator (KPIs)

- Avg Cost Per Acquisition

```
SELECT
    TO_CHAR(DATE_TRUNC('month', lbi.applied_date), 'YYYY-MM-DD') AS applied_dates,
    cast(ROUND(AVG(lbi.cost_per_acquisition), 2)as float) AS avg_cost_per_acquisition
FROM los_fact_loan_journey lfj
JOIN los_dim_business_insights lbi ON lfj.loan_id = lbi.loan_id
GROUP BY applied_dates
ORDER BY applied_dates
```

- Total Applications Submitted Over Time

```
SELECT
    TO_CHAR(DATE_TRUNC('month', lbi.applied_date), 'YYYY-MM-DD') AS applied_dates,
```

```
    cast(COUNT(DISTINCT lfj.application_id)as int) AS total_applications_submitted
FROM los_fact_loan_journey lfj
JOIN los_dim_business_insights lbi ON lfj.loan_id = lbi.loan_id
WHERE lfj.application_id IS NOT NULL
GROUP BY applied_dates
ORDER BY applied_dates
```

- Overall Conversion Rate

```
SELECT
    TO_CHAR(DATE_TRUNC('month', lbi.applied_date), 'YYYY-MM-DD') AS applied_dates,
    cast(COUNT(DISTINCT lfj.application_id) * 100.0 / NULLIF(COUNT(DISTINCT lfj.lead_id), 0)as float) AS conversion_rate_percentage
FROM los_fact_loan_journey lfj
JOIN los_dim_business_insights lbi ON lfj.loan_id = lbi.loan_id
GROUP BY applied_dates
ORDER BY applied_dates
```

- Average Approved Amount

```
SELECT
    TO_CHAR(DATE_TRUNC('month', lfj.loan_start_date), 'YYYY-MM-DD') AS loan_start_date,
    cast(SUM(lfj.approved_amount)as float) AS total_approved_amount,
    cast(ROUND(AVG(lfj.approved_amount), 2)as float) AS avg_approved_amount
FROM los_fact_loan_journey lfj
WHERE lfj.approval_status = 'Approved'
GROUP BY loan_start_date
ORDER BY loan_start_date
```

- Loan Disbursal Metrics Over Time

```
SELECT
```

```
    TO_CHAR(DATE_TRUNC('month', lfj.disbursement_date), 'YYYY-MM-DD') AS
disbursement_date,
    cast(SUM(lfj.disbursed_amount)as float) AS total_disbursed_amount,
    cast(ROUND(AVG(lfj.disbursed_amount), 2)as float) AS avg_disbursed_amount
FROM los_fact_loan_journey lfj
WHERE lfj.disbursed_amount IS NOT NULL
GROUP BY disbursement_date
ORDER BY disbursement_date
```

- Loan Declines Over Time

```
SELECT
    TO_CHAR(DATE_TRUNC('month', lbi.decline_date), 'YYYY-MM-DD') AS
decline_date,
    cast(COUNT(*)as int) AS total_loans_declined,
    cast(ROUND(AVG(lfj.credit_score_at_decline), 0)as int) AS
avg_credit_score_at_decline,
    cast(ROUND(AVG(lfj.income_at_decline), 2)as float) AS avg_income_at_decline
FROM los_fact_loan_journey lfj
JOIN los_dim_business_insights lbi ON lfj.loan_id = lbi.loan_id
WHERE lfj.approval_status = 'Declined'
GROUP BY lbi.decline_date
ORDER BY lbi.decline_date
```

Data Analysis Queries

Monthly Loan Applications & Approval Rate

Objective: To analyze monthly loan application volumes and approval rates, providing insights into overall application success trends and operational efficiency.

```
SELECT
  TO_CHAR(DATE_TRUNC('month', bi.applied_date), 'MM') AS application_month,
```

```
  cast(COUNT(fj.loan_id)as int) AS total_applications,
  cast(COUNT(CASE WHEN fj.approval_status = 'Approved' THEN 1 END)as int) AS approved_count,
  cast(ROUND(100.0 * COUNT(CASE WHEN fj.approval_status = 'Approved' THEN 1 END) / COUNT(fj.loan_id), 2)as float) AS approval_rate
FROM los_fact_loan_journey fj
JOIN los_dim_business_insights bi ON fj.loan_id = bi.loan_id
GROUP BY application_month
ORDER BY application_month;
```

Average Loan Amount and Interest Rate by Loan Type

Objective: To evaluate the average loan amount and interest rate for each loan type on a monthly basis, enabling assessment of product performance and pricing trends.

```
SELECT
  bi.loan_type,
  TO_CHAR(DATE_TRUNC('month', bi.applied_date), 'MM') AS applied_month,
  cast(round(AVG(fj.loan_amount),2)as float) AS avg_loan_amount,
  cast(round(AVG(fj.applied_interest_rate),2)as float) AS avg_interest_rate
FROM los_fact_loan_journey fj
JOIN los_dim_business_insights bi ON fj.loan_id = bi.loan_id
GROUP BY bi.loan_type, applied_month
ORDER BY applied_month
```

Top 10 Agents by Loan Applications Submitted

Objective: Identify the top 10 agents by loan applications submitted to evaluate agent performance and optimize resource allocation.

```
SELECT
```

```
    TO_CHAR(DATE_TRUNC('month', dbi.applied_date), 'YYYY-MM-DD') AS
applied_dates,
    CONCAT(dbi.agent_first_name, ' ', dbi.agent_last_name) AS agent_name,
    cast(COUNT(flj.application_id)as int) AS total_applications
FROM los_fact_loan_journey flj
JOIN los_dim_business_insights dbi ON flj.agent_id = dbi.agent_id
GROUP BY applied_dates, agent_name
ORDER BY total_applications DESC
LIMIT 10;
```

Loan Decline Reasons Breakdown

Objective: Analyze loan decline reasons to identify common rejection factors and enhance approval strategies while mitigating risk.

```
SELECT
    dbi.reason_name,
    cast(COUNT(flj.decline_id)as int) AS total_declines,
    cast(ROUND(AVG(flj.credit_score_at_decline), 2)as float) AS
avg_credit_score_at_decline,
    cast(ROUND(AVG(flj.income_at_decline), 2)as float) AS avg_income_at_decline
FROM los_fact_loan_journey flj
JOIN los_dim_business_insights dbi ON flj.decline_reason_id = dbi.reason_id
WHERE flj.approval_status = 'Declined'
GROUP BY dbi.reason_name
ORDER BY  total_declines DESC;
```

Revenue Generated by Region

Objective: Assess revenue generated by region to identify high-performing locations and optimize market expansion strategies.

```
SELECT
    dbi.state,
    cast(ROUND(SUM(dbi.revenue_generated), 2)as float) AS total_revenue
FROM los_dim_business_insights dbi
GROUP BY dbi.state
ORDER BY total_revenue asc
```

Agent Performance Analysis

Objective: Evaluate agent performance based on applications submitted, revenue generated, experience, and ratings to enhance productivity and incentive strategies.

```
SELECT
    TO_CHAR(DATE_TRUNC('month', dbi.applied_date), 'YYYY-MM-DD') AS
applied_dates,
    CONCAT(dbi.agent_first_name, ' ', dbi.agent_last_name) AS agent_name,
    dbi.performance_rating,
    dbi.experience_years,
    COUNT(flj.application_id) AS total_applications,
    cast(SUM(dbi.revenue_generated)as float) AS total_revenue
FROM los_fact_loan_journey flj
JOIN los_dim_business_insights dbi ON flj.agent_id = dbi.agent_id
GROUP BY applied_dates, agent_name, dbi.performance_rating, dbi.experience_years
ORDER BY total_revenue DESC
limit 10;
```

Application Decline Trends Over Time

Objective: To track monthly application volumes and analyze the decline rate percentage over time, helping identify trends in loan application rejections.

```
SELECT
    TO_CHAR(DATE_TRUNC('month', flj.disbursement_date), 'MM') AS month,
    cast(COUNT(flj.application_id)as int) AS total_applications,
    cast(COUNT(CASE WHEN flj.approval_status = 'Declined' THEN 1 END)as int) AS
total_declines,
    cast(ROUND(
        100.0 * COUNT(CASE WHEN flj.approval_status = 'Declined' THEN 1 END)
        / COUNT(flj.application_id),
    2)as float) AS decline_rate_percentage
FROM los_fact_loan_journey flj
GROUP BY month
ORDER BY month;
```

Customer Credit Score Distribution

Objective: Categorize customers based on credit scores to assess risk distribution and tailor lending strategies.

```
SELECT
    CASE
        WHEN cust_credit_score < 600 THEN 'Poor'
        WHEN cust_credit_score BETWEEN 600 AND 700 THEN 'Fair'
        WHEN cust_credit_score BETWEEN 700 AND 800 THEN 'Good'
        ELSE 'Excellent'
    END AS credit_category,
    cast(COUNT(*)as int) AS total_customers
FROM los_dim_customer_details
GROUP BY credit_category
```

Loan Disbursement Trends Over Time

Objective: Analyze loan disbursement trends over time to identify seasonal patterns and optimize lending strategies.

```
SELECT
    TO_CHAR(DATE_TRUNC('month', flj.disbursement_date), 'MM') AS month,
    cast(SUM(flj.disbursed_amount)as float) AS total_disbursed_amount
FROM los_fact_loan_journey flj
WHERE flj.disbursement_date IS NOT NULL
GROUP BY month
ORDER BY month;
```

Employment Status Distribution

Objective: To analyze the distribution of customers by employment status and identify their proportional share in the overall borrower base.

```
SELECT
    cust_employment_status AS employment_status,
    cast(COUNT(customer_id)as int) AS total_customers,
    cast(ROUND(100.0 * COUNT(customer_id) / SUM(COUNT(customer_id)) OVER (), 2)as float) AS percentage_share
FROM los_dim_customer_details
GROUP BY cust_employment_status
ORDER BY total_customers DESC;
```

Most Popular Loan Amount Brackets

Objective: Identify the most popular loan amount brackets to understand customer borrowing preferences and optimize loan product offerings.

```
SELECT
    CASE
```

```
        WHEN loan_amount_requested < 7000 THEN 'Less than 7K'
        WHEN loan_amount_requested BETWEEN 7000 AND 21000 THEN '7K-21K'
        WHEN loan_amount_requested BETWEEN 21000 AND 35000 THEN '21K-35K'
        ELSE 'More than 50K'
    END AS loan_bracket,
    cast(COUNT(*)as int) AS total_loans
FROM los_fact_loan_journey
GROUP BY loan_bracket
ORDER BY total_loans DESC;
```

Customer Feedback Score Distribution

Objective: Analyze customer feedback score distribution to measure satisfaction levels and identify areas for service improvement.

```
SELECT
    cast(feedback_score as text),
    cast(COUNT(*)as int) AS total_customers
FROM los_dim_customer_details
GROUP BY feedback_score
ORDER BY feedback_score DESC;
```